MA388 Sabermetrics: Lesson 12

Behaviors by Count - Umpire Tendencies

LTC Jim Pleuss

```
library(tidyverse)
library(knitr)
library(baseballr)
```

Lesson Objectives

1. Collect Statcast pitch-level data using the `baseballr` package.
2. Fit a LOESS model to estimate the probability of a called strike as a function of pitch location.
3. Visualize umpire strike zones across different counts using contour plots and faceting.
4. Assess whether umpires adjust their strike zone based on the count.

Today we're going to take our first look at pitch data in terms of where a pitch crossed (or didn't cross) home plate and the umpire's associated call.

```
get_statcast_pitches <- function(start_day, end_day, chunk_size = 5) {

  # Coerce to Date
  start_day <- as.Date(start_day)
  end_day   <- as.Date(end_day)

  # Create sequence of chunk start dates
  chunk_starts <- seq(start_day, end_day, by = paste(chunk_size, "days"))

  # Build data
  pitch_data <- map_dfr(chunk_starts, function(chunk_start) {

    chunk_end <- min(chunk_start + days(chunk_size - 1), end_day)
       statcast_search(
        start_date = chunk_start,
        end_date   = chunk_end
```

```
      )
  })
  return(pitch_data)
}


sc <-
get_statcast_pitches('2024-08-02','2024-08-13')
```

Do umpires adjust their strike zone based on the count? How might we go about gathering evidence?

We'll want to look at strike vs. ball calls throughout the strike zone and surrounding area, so let's build a data frame consisting of combinations of horizontal locations from -2 (two feet to the left of the middle of home plate) to +2 (two feet to the right of home plate) and vertical locations from the ground (value of 0) to five feet above the ground.

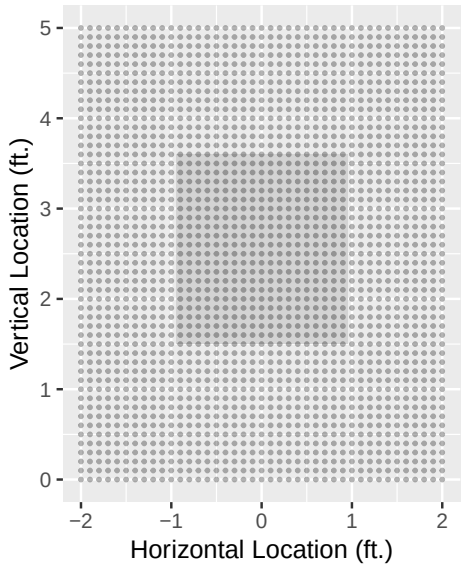
Here's a glimpse of the data frame we'll use.

```
umpires <-
  sc |>
  filter(description %in% c('ball','called_strike')) |>
  mutate(called_strike = description=='called_strike')
```

```
pred_area <- expand.grid(plate_x = seq(-2, 2, by = 0.1),
                         plate_z = seq(0, 5, by = 0.1))
```

Let's create a base plot with the strike zone outlined and then add our grid on top.

```
k_zone_plot <- ggplot(pred_area, aes(x = plate_x, y = plate_z)) +
  geom_rect(xmin = -0.947, xmax= 0.947, ymin = 1.5, ymax = 3.6,
            fill = "lightgray", alpha = 0.3) +
  coord_equal() +
  scale_x_continuous("Horizontal Location (ft.)", limits = c(-2, 2)) +
  scale_y_continuous("Vertical Location (ft.)", limits = c(0, 5))
k_zone_plot +
  geom_point(size = .5, alpha = 0.3)
```



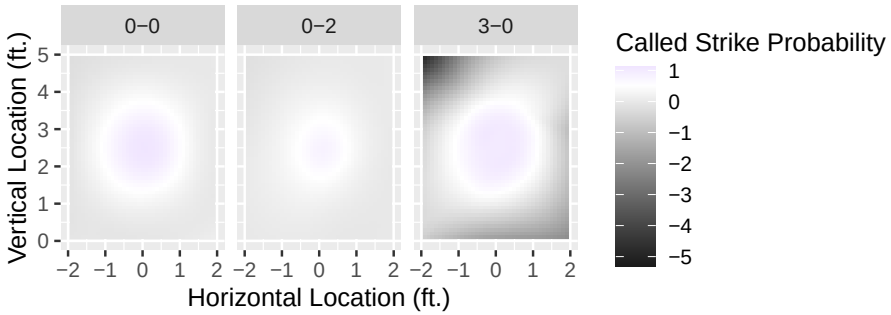
Digging into the Code!

What's happening here? Annotate the code.

```
ump_count_fits <- umpires |>
  filter(
    stand == "R",
    (balls == 0 & strikes == 0) |
    (balls == 3 & strikes == 0) |
    (balls == 0 & strikes == 2)) |>

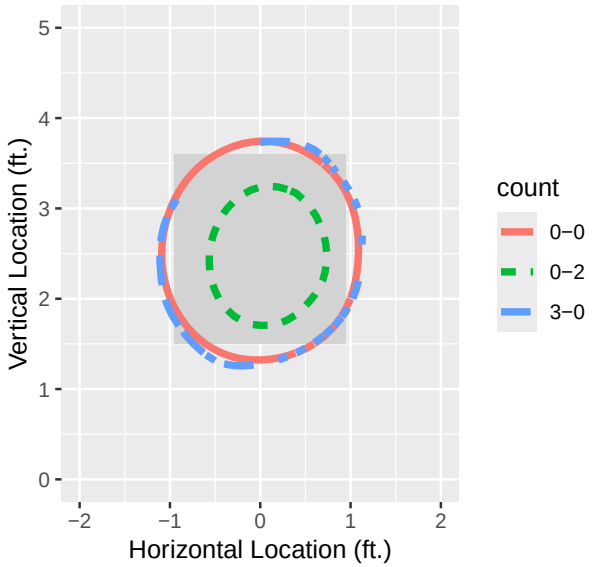
  mutate(count = paste(balls, strikes, sep = "-")) |>
  # filter(count=='3-0', plate_z<1,plate_x>1)
  split(~count) |>
  map(\(df) loess(called_strike ~ plate_x + plate_z,
             data = df,
             # span=.3,
             control = loess.control(surface = "direct"))) |>
  map(predict, newdata = pred_area) |>
  map(\(df) data.frame(fit = as.numeric(df))) |>
  map_df(bind_cols, pred_area, .id = "count")
```

```
k_zone_plot %+%
  ump_count_fits +
  geom_tile(aes(x=plate_x,y=plate_z,fill = fit),
            )+
  scale_fill_gradient2("Called Strike Probability",
                       low = "grey10",
                       high = "blue",
                       mid = "white",
                       midpoint = .5)+
  facet_grid(~count)
```



What's the problem with this legend?

```
k_zone_plot %+%
  filter(ump_count_fits,
         fit < 0.6 & fit > 0.4) +
  geom_contour(aes(z = fit, color = count, linetype = count),
               binwidth = 0.1, lwd = 1.5)
```



My Explanation: If we believe the count may affect an umpire's strike zone, we'll need to capture each unique count similarly to how we've previously captured the `STATE` when analyzing run expectancy. To do this we create a variable for count by combining the number of balls and strikes. We've narrowed our interest to right-handed batters and a neutral count, a hitter's count, and a pitcher's count. We then split the data on count.

We then fit a LOESS model that attempts to fit a locally estimated surface to the average strike call value (0 vs. 1) at each of our data points (pitches).

Next, we compute a set of predictions for each point on our grid for each count (one model per split in our original data frame). We then convert the numeric matrices returned by predict into a numeric vector called `fit` before turning those vectors into one dimensional data frames. These data frames are then added to the `pred_area` data frame with the `bind_cols()` command, effectively adding the likelihood of a strike call to every `(px,pz)` combination for every count (0-0, 0-2, 3-0).

Has this gotten better since the pitch clock? Hmm...